

基于 AI 工具的工程化

Engineering with AI

2026.05.29 + 2026.06.05

by 林少彪

为什么别人可以 YOLO，而“我”不敢？



Lawyer

I learned from law school.



Doctor

I learned from med school.



Vibe Coder

Learning is old-fashioned.
I prompt... and pray.

“ Source: YOLO mode! 😂 ”

YOLO `/'jəʊ.ləʊ/`: You Only Live Once, 字面意思为“只活一次, 及时行乐”。在 AI Coding 语境中可以有多种理解:

- **工具使用**: Coding Agent 的 YOLO mode
- **实践作风**: 通常指一种非常激进、低约束、强依赖 AI 自动生成并部署代码的开发方式。
 - “别想太多, 直接干”
 - “先跑起来再说”
 - “让 AI 自动改, 出了问题再修”
 - “accept all”
 - “ship first”

目录

1. 业界前沿实践：Harness Engineering

2. 我们的现状与挑战

3. 多 Agents 系统

I. 业界前沿实践：Harness Engineering

1.1 Harness Engineering 简介

- harness `/ˈhɑːnɪs/` : n. 挽具; v. 控制并利用, 即“驾驭”。
- Harness Engineering 即是“围绕如何驾驭 AI 展开的工程实践”。

Prompt Engineering

↓ 让模型生成更好的单次输出

↓

Context Engineering

↓ 为 AI 设计它能“看到、理解、记住、推理”的信息系统

↓

Harness Engineering

↓ 让 AI 在系统中稳定、可控、可验证地工作

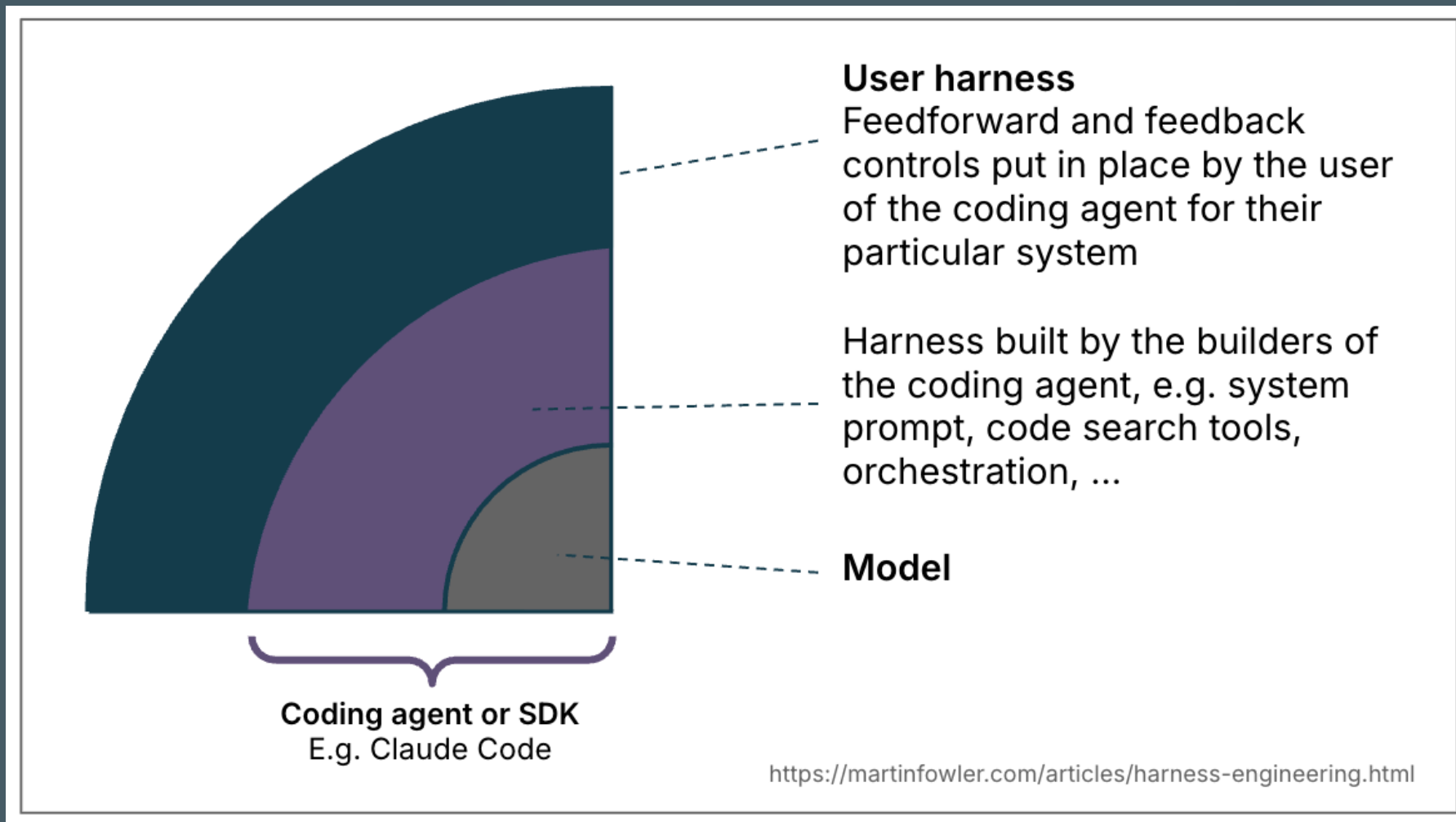
?

1.2 思维模型 (Mental Model) 新鲜度 2026.04.02

“ [Harness engineering for coding Agent users \(martinfowler.com\)](https://martinfowler.com/harness-engineering-for-coding-agent-users) ”

“harness”在实践中分三层：

1. **模型层**：围绕原始 LLM 的工具与脚手架。
2. **Agent 构建层**：开发 Agent 所采用的内置 system prompt、检索、编排方案。
3. **Agent 使用层**：作为用户为 Agent 配置外围约束。★



Harness 分层

“ Permission: Can I use one of your illustrations or photographs?

”

1.2.1 前馈与反馈

- **指引 - 前馈控制 (Guides, feedforward controls):** 编码约定、架构原则、设计文档、代码 review 规则等，在行为发生前就设定好。
- **感应器 - 反馈控制 (Sensors, feedback controls):** 测试、监控、评审等，在行为发生后进行检测。

1.2.2 计算与推断 (Computational vs. Inferential)

类型	成本 / 速度	确定性	例子
计算型	毫秒到秒，成本低	高确定性	单元测试、lint、类型检查
推断型	秒到分钟，成本高	非确定性	AI 代码评审、语义约定检查、文档检查

1.2.3 掌舵回路 (The Steering Loop)

人类的核心工作是通过迭代 harness 来掌舵 agent:

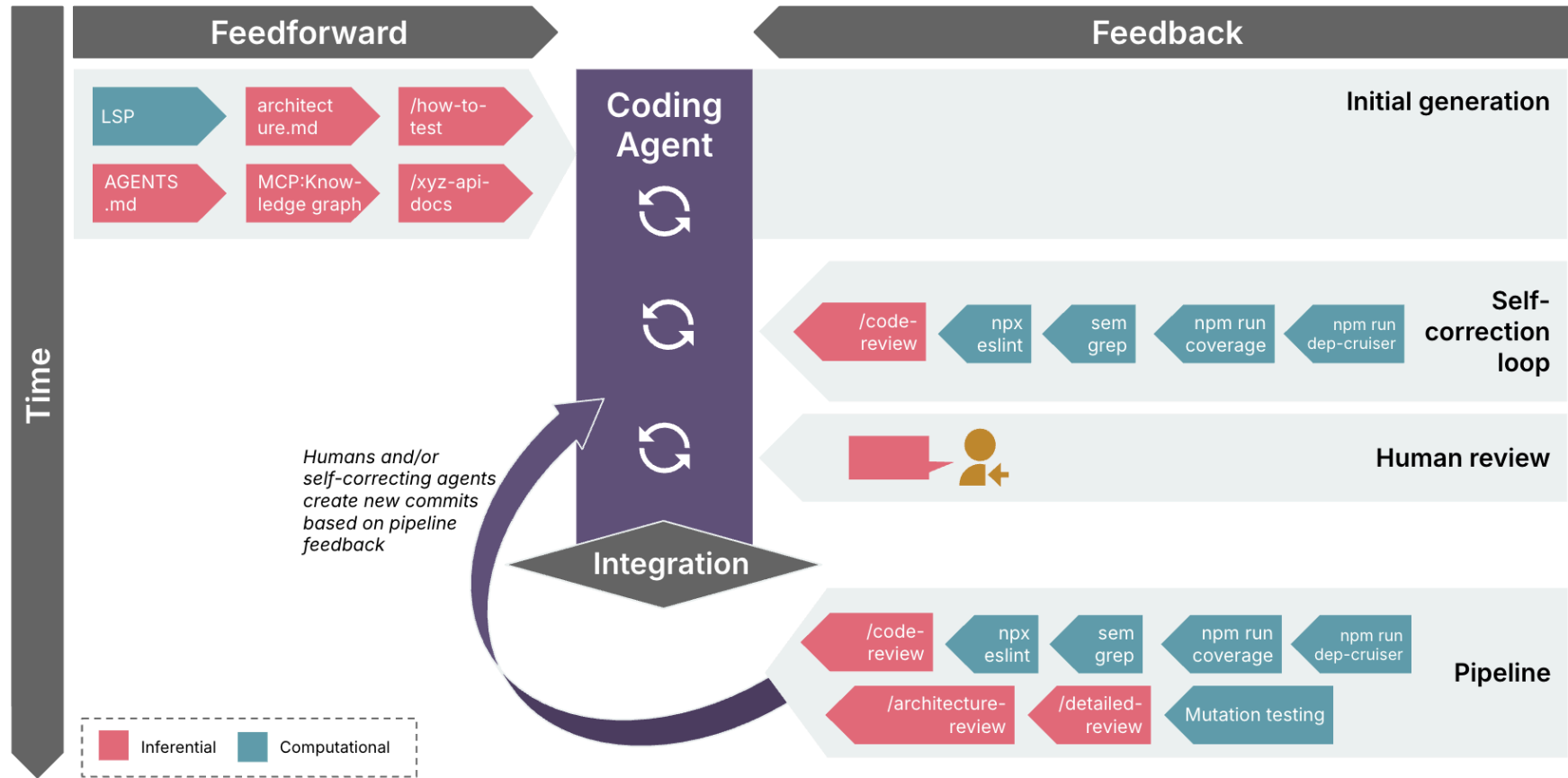
发现问题 → 是否重复出现? → 优化 guide/sensor → 再运行

1.2.4 质量左移 (Keep Quality Left)

需求 → 开发 → 提交代码 → CI → 测试 → 发布 → 生产环境
↑ 左边 右边 ↓
越早 越晚

“质量左移”的理念源于传统 CI/CD，AI 推理式检查出现后，也需要把这些新的反馈机制分布到整个开发生命周期里。

Examples: Feedforward and feedback in the change lifecycle

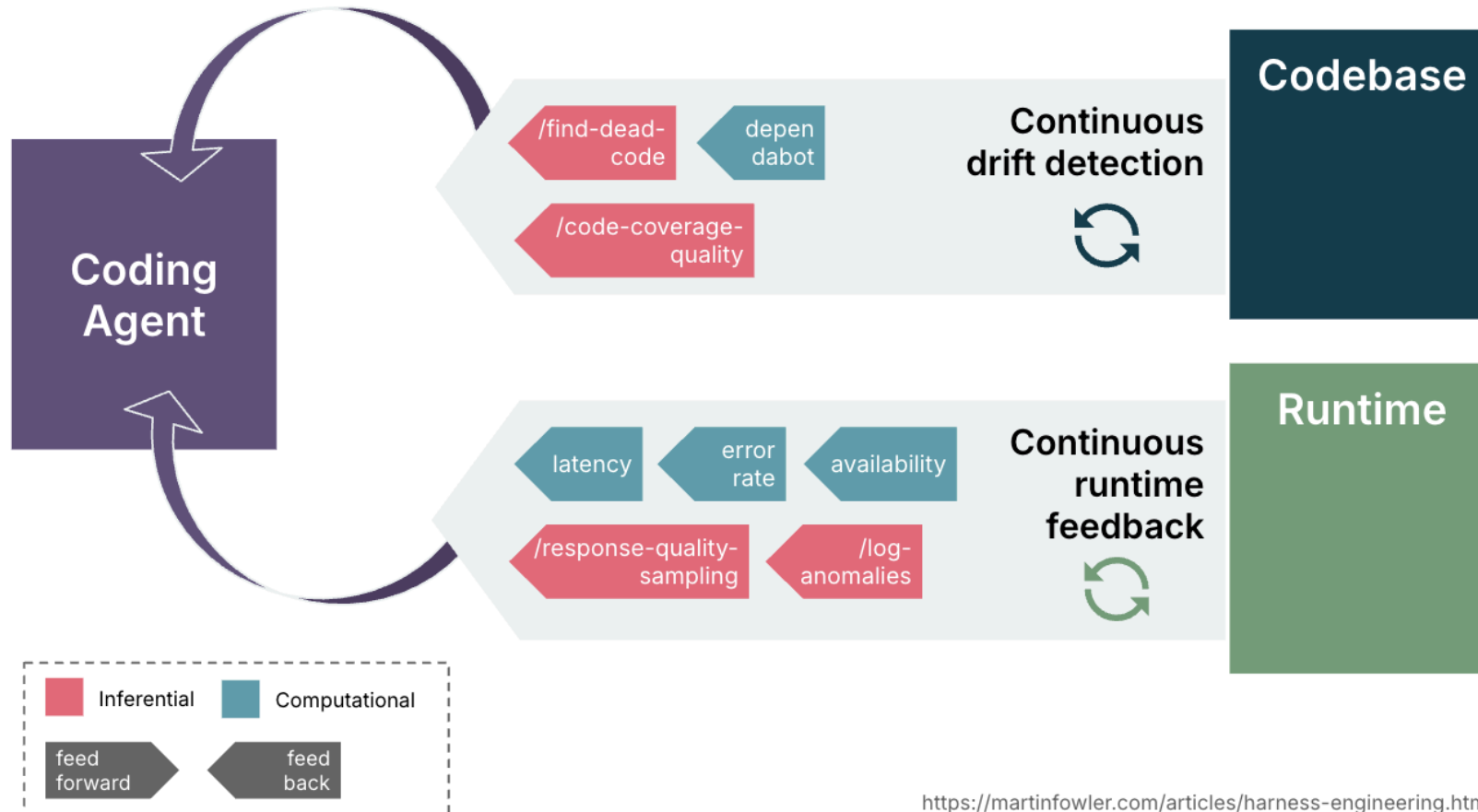


<https://martinfowler.com/articles/harness-engineering.html>

变更迭代生命周期

“ Permission: Can I use one of your illustrations or photographs? ”

Examples: Continuous drift and health sensors



持续漂移与健康感应

“ Permission: Can I use one of your illustrations or photographs?

”

1.2.5 治理维度

A. 可维护性 Harness - 最成熟

调节内部代码质量：重复、复杂度、测试覆盖、架构漂移、风格。

- 计算型 **sensor** 得益于大量传统工程工具，可稳定发现结构问题。
- 推断型 **sensor** 可处理语义问题（语义重复、冗余测试、过度设计），但成本高且不稳定。

B. 架构适应度 (fitness) Harness

调节架构适应度：性能、可观测性、安全态势、可扩展性。比如：

- 用 skill 描述性能预算 + 性能测试把回归反馈给 Agent。
- 用 skill 约束日志/可观测性 + 调试 skill 让 Agent 反思日志质量。

C. 行为 Harness - 最不成熟，仍是开放问题

调节应用是否“真的满足用户功能需求”。当前常见的模式脆弱：

- 前馈：功能规格，常常离完备描述还有很大差距。
- 反馈：依赖 AI 生成测试 + 人工测试。

这种 harness 依赖 AI 生成测试，但是 AI 可以：

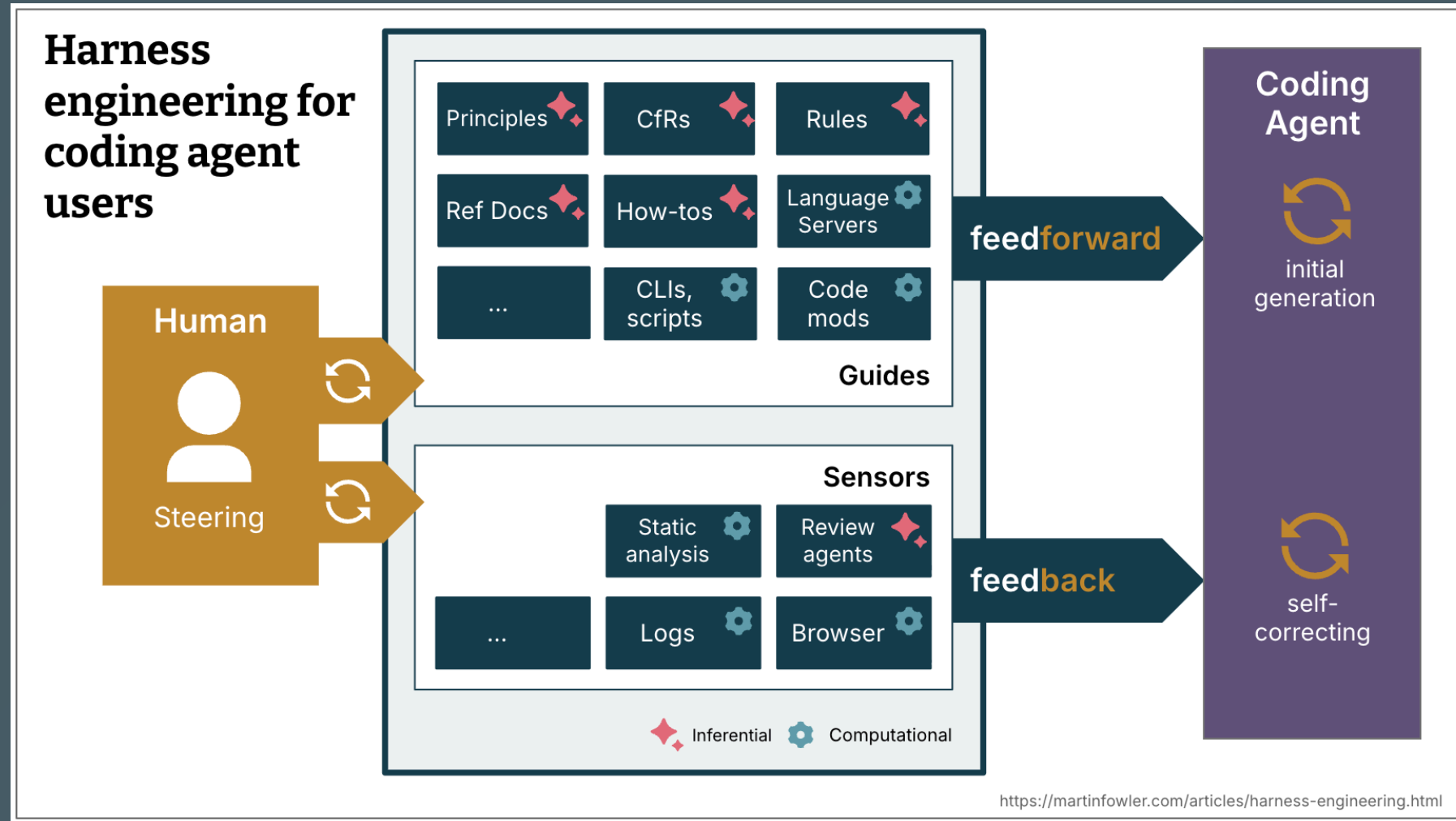
写 bug → 再写一个“测不出 bug 的测试”

所以结果仍然需要人工评审与手工测试。

1.2.6 “可驾驭性” Harnessability

因素	影响
强类型系统	类型检查器天然成为高质量 sensors
清晰模块边界	架构规则更可强制执行
强约束框架	隐藏大量实现细节，减少 Agent 发明空间
成熟测试集	行为 sensors 已具备
约定被写下来	前馈 guides 已存在

- 新项目可从第一天就“为 harness 而设计”。
- 遗留系统更难：越需要 **harness** 的地方，往往越难补齐 **harness**。



思维模型 (mental model)

“ Permission: Can I use one of your illustrations or photographs?

”

1.2.7 启示

- 在把大任务交给 Agent 前，先问：是否存在一套可运行、行为错误时会失败、且不是由同一个 Agent 生成并自评的测试集？有则行为 harness 已具备；没有则先建设它。
- **人类角色**：资深开发者天然具备 Agent 默认缺失的能力，harness 能外化其中一部分，但不是全部。
 - 社会责任感（提交可追责）。
 - 对糟糕代码结构的直觉反感。
 - 知道哪些约定是关键、哪些只是习惯。
 - 对组织现实的对齐：哪些技术债可接受，为什么。
 - 节奏控制：小步迭代给思考留空间。
- **目标不是消灭人类输入，而是把人类精力放到最关键的位置。**

1.3 Everything Is A Ralph Loop

新鲜度 2026.01.17

“ [everything is a ralph loop \(ghuntley.com\)](http://everything_is_a_ralph_loop.ghuntley.com) ”

Ralph Loop 一词来自动画片《辛普森一家》里的角色：Ralph Wiggum，他是个著名的“笨蛋但乐观执着”的角色。经典台词包括：

“ Me fail English? That’s impossible.
我英语挂科？那是不可能的。——故意语法错误 ”

1.3.1 Ralph Loop 理念

- 要把 AI 当成可反馈、可调优、可编排的自动化循环系统。
- 叫 Ralph Loop 其实是一种自嘲：这个方法看起来“蠢”得像 Ralph Wiggum 一样。

“不要相信 *Agent* 的长期智能”

而是：

“让 *Agent* 每次只做一小步，然后不断重启”

传统 Agent 思路	Ralph Loop 思路
维持长期上下文	每轮 fresh context
强调规划能力	强调 brute force iteration
智能 orchestration	dumb persistence (简单循环持续执行)
memory-heavy	filesystem-as-memory
一个强大的 Agent	很多短生命周期 Agent

1.3.2 极简循环

Goal → Context → Execute → Verify → Repair → Repeat

```
while failed; do  
    opencode run -m opus-4-7 "$PROMPT"  
done
```

1.4 OpenAI Codex 新鲜度 2026.02.11

“ 工程技术：在智能体优先的世界中利用 Codex (openai.com) ”

1. *Humans steer, agents execute*

- 人类负责：目标、架构、约束、验证机制、feedback loop
- Agent 负责：写代码、写测试、修 bug、改 CI、写文档、review PR

2. 软件工程的重点变了：从“写代码”变成“设计系统”

- 有没有足够上下文、明确约束、自动验证、高质量反馈
- 系统是否“对 Agent 可理解”

3. “*Agent legibility (可理解性)*” 是核心

- 代码库必须对 AI Agent “可读、可推理、可导航”
- 一切知识“repo 化”

4. ***Harness*** 的本质是反馈回路 (***feedback loops***), OpenAI 构建了大量自动反馈系统:
- UI 可观测: 启动 app、打开浏览器、操作 UI、看截图、分析 DOM、自动复现 bug
 - 服务可观测: logs、metrics、traces、spans
 - 自动 review loop: 自 review、调其他 agent review、修 review comment、重跑测试、build、再 review
5. 约束比自由更重要: ***Agent*** 越强, 越需要强约束
6. 技术债本质上是 Agent entropy (熵增)
- 即所有会 ***干扰 Agent 产生正确代码的因素***: 烂代码、不一致风格、YOLO probing、随意 schema 等都会被无限复制。
 - ***垃圾回收是必要的***: 定期扫描 repo、发现坏模式、提 refactor PR、修复 drift。
7. 代码合并原则发生了重要的变化:
- 以前: 低吞吐 → 谨慎 merge
 - 现在: 高吞吐 agent → 修复成本低

```
AGENTS.md
ARCHITECTURE.md
docs/
├── design-docs/
│   ├── index.md
│   ├── core-beliefs.md
│   └── ...
├── exec-plans/
│   ├── active/
│   ├── completed/
│   └── tech-debt-tracker.md
├── generated/
│   └── db-schema.md
├── product-specs/
│   ├── index.md
│   ├── new-user-onboarding.md
│   └── ...
├── references/
│   ├── design-system-reference-llms.txt
│   ├── nixpacks-llms.txt
│   ├── uv-llms.txt
│   └── ...
├── DESIGN.md
├── FRONTEND.md
├── PLANS.md
├── PRODUCT_SENSE.md
├── QUALITY_SCORE.md
├── RELIABILITY.md
└── SECURITY.md
```

将代码仓库视为记录系统

1.5 Anthropic Claude Code 新鲜度 2026.05.14

- “
- [How Claude Code works in large codebases: Best practices and where to start \(claude.com\)](#)
 - [Claude Code 最佳实践 \(claude.com\)](#)
- ”

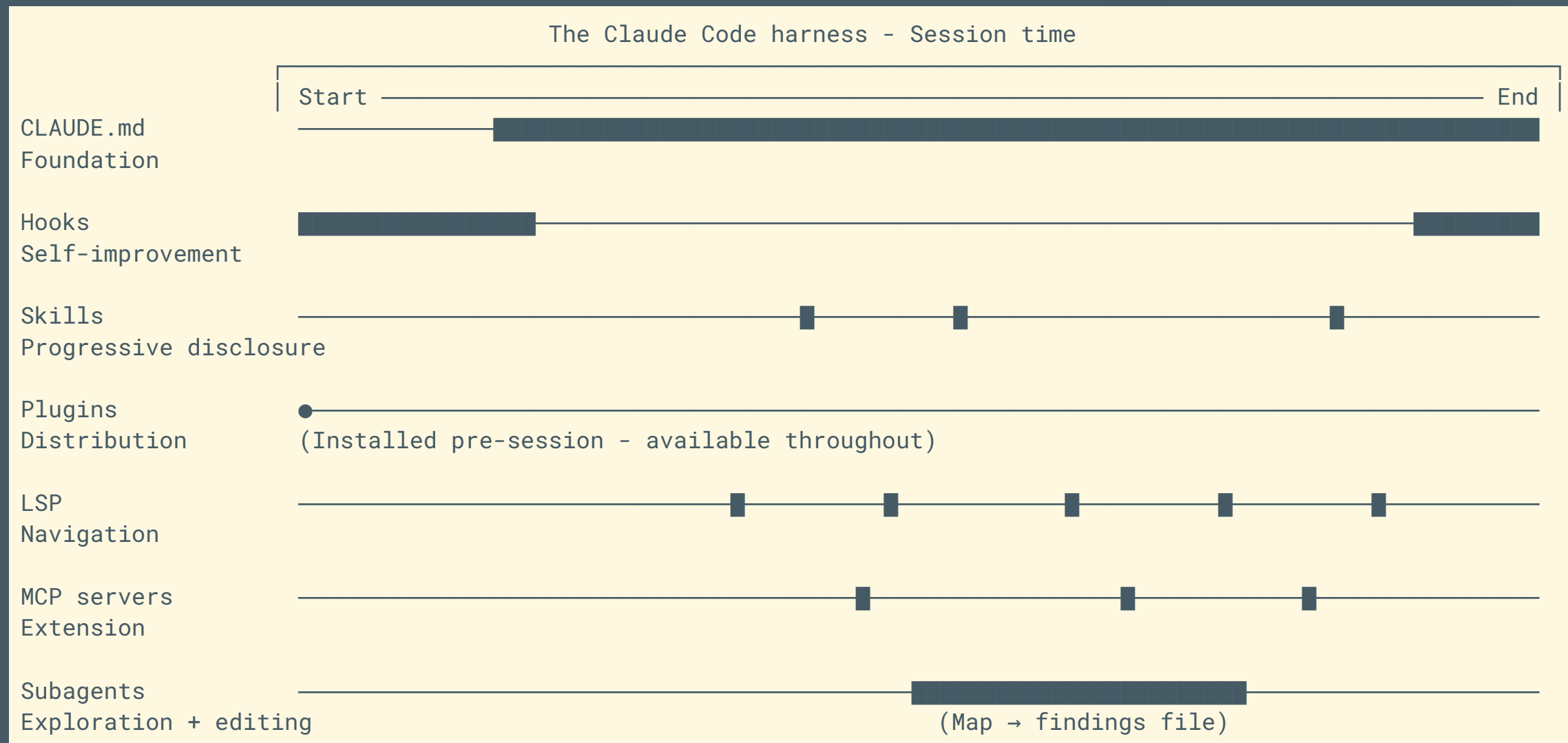
1.5.1 Claude Code 如何在大型 codebase 中导航

大型代码库里“实时探索”比“静态索引 (RAG)”更可靠，因为静态索引容易过时。Claude Code 是像真人工程师一样：

- 遍历文件系统
- grep 搜索
- 跟踪引用

- 阅读文件
- 动态探索代码结构

1.5.2 Harness 跟模型能力一样重要



1.5.3 三种成功的 Claude Code 管理范式

A. 让代码库“可导航”

1. 保持 `CLAUDE.md` 文件简洁且分层
2. 在子目录中初始化 `CLAUDE.md`，而不是在仓库根节点 (针对 monorepo 或大型项目)
3. 为每个子目录设定测试范围和 lint 命令
4. 使用 `.ignore` 文件排除生成文件、构建工件和第三方代码
5. 当目录结构没有章法时，使用 `CLAUDE.md` 指明代码库模块图谱
6. 使用 LSP，以便 Claude Code 通过符号而非字符串进行搜索

B. 定期积极维护 `CLAUDE.md`

随着模型智能和 Agent 能力的提升，以往的 `CLAUDE.md` 可能会变得冗余甚至误导。定期审查并更新 `CLAUDE.md`，确保它反映当前代码库的结构和最佳实践。

C. 明确责任归属：Claude Code (Agent) 配置的管理与推广 ★

- 阶段 01 — 静默投入（仅有核心负责人搭建基础能力）
 - 时机：开放使用之前
 - 负责人逐步搭建各项基础设施
- 阶段 02 — 正式发布落地（小规模首批开发者启用）
 - 时机：发布首日
 - 基础设施筹备完成
 - 首批开发者验证产品具备生产力
- 阶段 03 — 规模化推广渗透（核心用户向外辐射，更多团队逐步接纳落地）
 - 时机：发布之后
 - 口碑在各个团队之间传播扩散
 - 使用这套工作框架（harness）的人群持续增长

II. 我们的现状与挑战

2.1 三个比较有代表性的行业例子

1. [CCC - Claude's C 编译器](#)：Claude Code 在无人直接写代码的前提下用 Rust 实现了完整的 C 编译器。
2. [使用 Rust 重写 Bun](#)：一个超过 100 万行代码变更的 MR 被一次性合并。
3. [飞书 CLI 工具](#)：在数周内开发交付覆盖十几个业务域、200+ 命令的 CLI。

问题：这些项目除了都用 *coding agents* 加速开发之外，还有什么重要条件让它们得以快速验收？

2.2 CCC (Claude C Compiler) 项目

“ [Building a C compiler with a team of parallel Claudes \(anthropic.com\)](https://anthropic.com) ”

2.2.1 一些特点

- 人类不直接参与开发过程
- 多 agents 并行开发
- 耗时大约 2 周
- 模型 Opus 4.6
- 2,000 次 Claude Code 会话

- Token 成本 \$20,000
 - 20 亿输入 tokens
 - 1.4 亿输出 tokens
- 留存问题
 - 不支持 16 位硬件
 - 不包含汇编器、链接器，
 - 仍然有一些无法编译通过的项目
 - 编译性能较差，比不上 GCC 性能最差的模式
 - 代码质量仍然比不上人类高级工程师
 - 到项目后期，实现特性及修复 bug 经常出现整体质量回归

2.2.2 Harness 设计

人类设计反馈回路（依然是个 Ralph Loop）：

- 沙箱环境（Docker 容器）
- [高质量现成测试集](#)
- CI 门禁，防止质量回归
- 使用任务“锁”避免 agents 做同样的任务
 - 用需求文档作为锁，每个 agent 开发前先占用对应的需求文档
- 增量调试
 - GCC 作为预言机，每一轮先编译大部分 Linux 内核代码
 - CCC 编译剩余模块
 - 总体构建失败时则得到新的测试用例，直到 CCC 能编译整个 Linux 内核为止
- 分角色执行：一部分 agents 做代码审查、重构，维持代码质量

Ralph Loop:

```
#!/bin/bash

while true; do
  COMMIT=$(git rev-parse --short=6 HEAD)
  LOG="agent_logs/agent_${COMMIT}.log"

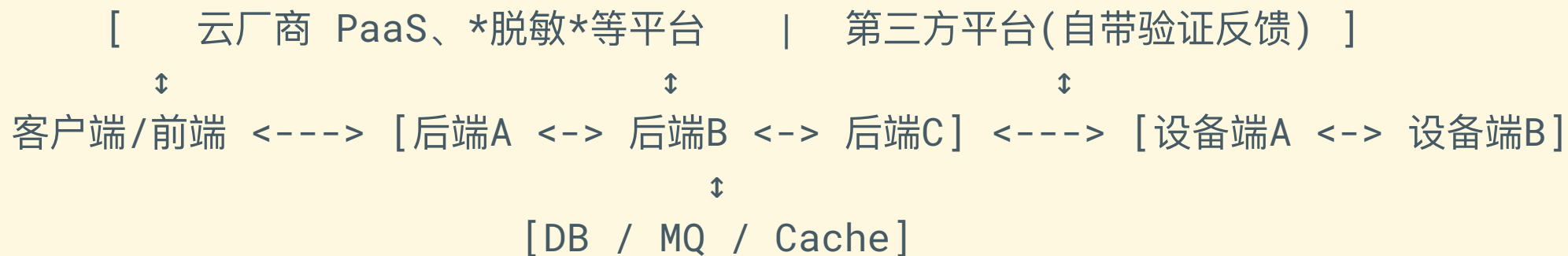
  claude --dangerously-skip-permissions \
    -p "$(cat AGENT_PROMPT.md)" \
    --model claude-opus-X-Y &> "$LOG"
done
```

2.2.3 模型能力的影响

- Opus 4 几乎无法产生一个功能正常的编译器。
- Opus 4.5 是第一个突破门槛的模型，最终生成的编译器能够通过大型测试集，但它仍然无法编译任何真正的大型项目。
- Opus 4.6 完成了一个功能完整的 C 编译器，能够编译 Linux 内核、PostgreSQL 这种大型生产级软件。

2.3 我们的现状

1. 当前阶段主要工作仍然是处理新需求，方案设计、功能验收离不开人类确认（大部分企业级应用的特点）
2. 系统链路长，反馈回路涉及的边界多，自动化验收一个完整功能的成本较高（硬件项目、复杂系统的特点）



3. 工作流、反馈回路有待建设

2.4 TODO List

2.4.1 前馈控制 - Guides

2.4.1.1 团队共用 Agents 配置

1. 自动安装脚本

- User 级通用配置
- Repo 级通用配置：项目特定配置维护在对应仓库中

2. Coding Agents 适配：OpenCode ★, Codex, Claude Code, Copilot, Cursor, Gemini

3. Spec Driven - OpenSpec

4. Skills: 自制之前先看看开源社区 ([GitHub Skills](#)) 有没有现成可用的

- http-api-design (http-specs 做成 skill 的好处是天然跨技术栈)
- grpc-api-design
- db-schema-design
- architecture-design
- code-refactor
- architecture-refactor
- code-review: 组合复用其他 skills
- ...

2.4.2 反馈控制 - Sensors

本地 hooks 适合快速反馈、节约成本，云端流水线作为最终的门禁：

- Linters
- Formatters
- 单元测试
- AI Code Review: 使用 review skills
- 平台整体 Changelog 自动聚合
- ...

2.4.3 主动发现

- Backlog 优先级评估
- 定期自动化测试 -> 缺陷自动提报
- 定期扫描代码 -> 缺陷/重构/性能优化自动提报
- 架构 review -> 漂移/安全/性能问题自动提报 -> 更新架构 skill

2.4.4 workflow建设与优化

- 周报（尽量做到飞书内闭环）
- 跨团队生产问题调试：研发日志聚合、调试流程优化（麻烦）
- 运维成本优化：日志规范、数据存储规范...

- AI Agent role based 需求评审
 - 产品需求文档评审
 - 开发评审 (后端、前端、设备端)
 - QA 评审
 - 架构评审
 - 安全评审
 - 合规评审
 - SRE 评审
 - 工作量预估

III. 多 Agents 系统

仍然在快速变化中的编排工具

- [CrewAI](#)
- [OpenAI Symphony](#)
- [Claude Dynamic Workflows](#)
- [ByteDance DeerFlow 2.0](#)
- [OpenClaw](#)
- [Hermes Agent](#)